Why DFBB Stalls on Century — Diagnosis and Plan

Sep 25, 2026 · @j

The search never left the opening: 72,785 states at branching 12–21 covers about four plies of a 40-turn plan, so the incumbent never tightened and no bound improvement could have helped.

Diagnosis

The measured numbers say the search is not close. At branching 12, $12^4 \approx 20{,}000$ and $12^5 \approx 250{,}000$, so 72,785 states searched in 60 seconds is an exhaustive sweep of roughly the first four or five turns. No terminal state was ever reached, so the incumbent `b` could never tighten.

Quantity	Value	Consequence
Unpruned depth (`b` − `h`)	41 − 11 = 30 plies	Nothing is cut until 30 turns deep
Branching after the payment rule	~12	Reaching 30 plies needs ~10^{32} states
States searched in 60 s	72,785	About 4–5 plies
Gain from the stronger `h`	+3 turns, 3–6% more pruning	Exactly what a 3-ply shift is worth

This reframes the problem. The bound is not slightly weak; the gap between what DFBB needs and what it gets is roughly 25 orders of magnitude. Any change that moves `h` by single digits is irrelevant on its own, and so is implementation speed: a 100× C++ speedup buys about two plies at branching 12. Hypothesis 6 in the handoff can be dropped.

The two quantities that matter are **depth** and the **incumbent**, in that order, because cost is branching raised to (`b` − `h`).

What the paper actually does

Three differences from Churchill & Buro (2011), all checked against the PDF.

1. The goal is a given resource vector. Their `GetGoal(B, ts, te)` extracts a concrete set of units and structures from a professional replay. Both admissible bounds depend on knowing it: LandmarkLowerBound sums the durations of the known prerequisite chain, ResourceGoalBound sums the known total cost. Breadth limiting comes from it too, as in "the goal has two fighters, so at most two barracks."

Century's goal is "6 golems." Which six, and which merchant cards are needed to afford them, is the search itself. `owed` prices the destination using the cheapest remaining golems, which is admissible, but charges nothing for building the engine that gets there — and that is where most of the 40 turns go. This is a structural gap in `h`, not a tuning gap.

2. They never solve a whole game. Algorithm 2 is the tell: when the full goal misses the time limit, they cut the build order into 15 or 30-second increments and solve each in turn, executing before planning the next. Figure 1 shows optimal search (K=1) on one late-game goal of two carriers consuming up to about 10^9 nodes, and they ship K=2 macro actions instead. Their headline result is short-horizon subgoals matched to professionals, not proven-optimal full games.

3. "Earliest execution" removes the timing dimension. They argue that hoarding resources cannot reduce makespan, so an optimal plan issues every action as soon as it is legal. That lemma kills the wait branch and justifies fast-forwarding. Century inverts it: hoarding crystals toward a golem is mandatory, and when to Rest is a real decision. That is the source of the cycles and the 10 transposition hits per state.

Correction to handoff item 7. The paper's macro actions are only a repetition cap K allowing an action to double up, and its action pruning is the goal-derived breadth limit. There is no dominance pruning in it. Both mechanisms need a known goal, so neither transfers unchanged.

Ranked changes

Ranked by effect on branching^(`b` − `h`). The first two attack that exponent directly; the rest are worth doing but cannot rescue the search alone.

#	Change	Label	Why it moves the needle
1	Solve to the **next golem**, not to six, and chain the subproblems	Heuristic overall, safe within each subproblem	Depth falls from ~40 to ~6. The only change that attacks the exponent. This is the paper's Algorithm 2
2	Better incumbent: beam search over the bot's move ordering	Safe	Cutting `b` from 41 to 30 is worth 11 plies, about 10^{11} states. Roughly ten times cheaper to build than a better `h`
3	Bound counting **forced zero-gain turns**: *n* cards allow at most *n* plays between Rests, and acquires cost turns too	Safe	The closest true analogue to LandmarkLowerBound. Should lift `h` from ~11 toward ~20
4	Acquire payment as a deferred sub-decision, or keep the yellow/green rule	Heuristic	73% of moves are payment variants of one underlying choice. Halves branching at every ply
5	IDA* with the f-limit driven by `h`	Safe	Decouples pruning from `b`, so items 3 and 6 pay from the first move instead of at depth 30
6	Max of `h` over all four value scales; per-colour bounds	Safe, cheap	A few percent each. Do them, but expect nothing on their own
7	Macro actions: play upgrades fully, Rest-then-play, max trade repeats	Heuristic	Cuts depth, which is what matters. Needs the tiny-board check

Item 1 changes what is being proved: each subproblem's optimum is provable, but the chain of them is not globally optimal. Keep the two claims separate in any write-up.

Recommended target: a provable gap

Stop aiming for a proven optimum on full games and report a bracket per seed instead.

- **Lower bound:** `h` at the root, strengthened by items 3 and 6. Currently 11 + claims.
- **Upper bound:** the best plan any method finds, from beam search or the chained subproblems.
- **The gap** between them is the headline number, narrowing as either end improves.

Why this is the right target:

- Solo Century is numeric planning with no prerequisite structure, reversible resource flow and a ~40-step horizon. Exact search is plausibly out of reach, and the paper's own scaling curve suggests StarCraft's would be too without a given goal.
- The bound validation already done supports it: `h` never exceeded true remaining turns across 42,749 states and 98 proven-optimal states, which is what makes the lower end trustworthy.
- It is an anytime result. Every improvement to either bound shows up immediately, rather than after the search finally terminates.

A reasonable first milestone: for seeds 0–9, report `h` at the root against the best plan found, and show the gap closing as items 2 and 3 land.

Next steps

In order, each with the check it must pass before the next.

1. **Commit the potential bound** in `solver/bound.py`. Tests are finished and passing.
2. **Beam search for the incumbent.** Reuse the bot's move ordering with a width of a few hundred. Check: `b` drops below the current 34–50 on seeds 0–9, and each plan replays to a real win.
3. **Forced zero-gain turn bound.** Check: still never exceeds true remaining turns on the 42,749-state set and the 98 proven-optimal states, and root `h` rises measurably.
4. **Next-golem subproblems.** Solve to one claim, execute, repeat. Check: each subproblem terminates within budget, and the chained plan beats the beam-search plan.
5. **IDA*.** Check: agrees with DFBB and the tiny-board brute force on every board both can run.
6. **Macro actions and remaining bounds.** Check after each: `solver/tiny.py` brute force still agrees; log nodes and runtime in `solver/bench.py` to see what each one bought.

Constraints from the handoff hold throughout: the engine stays authoritative, changes to `engine/` are opt-in flags defaulting to old behaviour, and every speed-up is labelled safe or heuristic before it is trusted.